This is the documentation for the project “Wi-Fi based microphone for conference and orientation of camera to speaker” and how to make the program work.

**Layout** -

The layout is such that the ESP8266 is configured as the AP - “main” network and the devices are connected to it and simultaneously connected to the servo-controlled camera.

The speaker module and the anchor modules are ESP32 – S3 as they support FTM

The main module- the speaker is also configured as the FTM initiator and connected to the “main” network which sends the voice data to the Laptop/Server and the distance information to the ESP8266.

There are 2 Anchor points which act as FTM responders to give the distance to the FTM initiator.

The laptop is also connected to the “main” network and the camera feedback is connected to it.

**Things to consider/change -**

1. The anchors are configured in such a way that they don’t need to be connected to a network or act as an AP, you only need each anchors MAC Address, which is different for every ESP32–S3 module (unless customized) and must be then changed in the final code of the speaker module.
2. The Laptop/Server acts as a middleman for all the data processing, for that you have to know the IPv4 address of your device and then also change that in the final code.
3. In the ESP8266 code, the camera should be placed in the dead centre of the 2 anchors and the max distance in the code should be changed to the distance in cm between the 2 anchors.

**Initialization** -

After all the necessary changes made to the code, upload each code to the respective modules, the “main” network code to the ESP8266 module, the anchor code to the 2 Anchor modules, the Final code with the correct MAC address of the anchor and the IPv4 address of the device uploaded to the speaker module(the speaker pins are configured as stated within the code, if different configuration then just change the gpio pins stated)and the python code open on the Laptop/Device.

After all the uploading of the code, first make sure of the order in which they are powered on.

First the 2 anchors fixed at the corners with the required distance between them and powered on.

Second the ESP8266 board and the camera feedback connected to the Laptop/Device

Third the Laptop/Device connected to the “main” network and the python instance running.

Fourth the speaker module finally powered up at the very last, which then should send FTM request to the anchors and receive them and send the distance calculated to the ESP8266 board. Also connected to the “main” network which sends voice data through UDP and is received by the laptop and played by the python instance.

**Future hopes of improvement/optimization and features -**

Make the Localization more accurate by implementing both RSSI and FTM together.

You can use physical improvements like a reflector or absorber behind the anchor points.

Add more robust way to add more speaker modules to the system and easy way to start and stop talking through the speaker.

If you follow everything correctly with the right components then it should work fine (obviously as a prototype) and I hope maybe you can improve upon it or make something new by taking inspiration from it.

THANK YOU!!!

PRASUN KRISHNA